



NATURAL LANGUAGE PROCESSING

AULA 6



Prof. Luciano Frontino de Medeiros



CONVERSA INICIAL

O processamento de linguagem natural teve avanços significativos com o advento do paradigma de programação de redes neurais artificiais de aprendizado profundo (*deep learning*). O objetivo desta aula é conhecer um pouco sobre *deep learning* e suas ferramentas para desempenho em tarefas de NLP. Para isso, é importante conhecer sobre o papel do *perceptron* multicamada para a construção de redes mais complexas. Também é necessário conhecer um pouco sobre representação vetorial (algo já abordado de forma pontual em conteúdos anteriores) de palavras para processamento em redes neurais artificiais. A partir daí, busca-se conhecer as redes neurais convolutivas (CNN) e também as redes neurais recorrentes (RNN), que permitem trabalhar com memória e ativação lateral. Com base nas RNN, outros tipos de arquiteturas são bastante utilizados, tais como a memória de longo prazo (LSTM) e a unidade recorrente com porta (GRU).

TEMA 1 – NLP E DEEP LEARNING

Arquiteturas e algoritmos de aprendizado profundo (*deep learning*) têm proporcionado avanços consideráveis em áreas como visão computacional e reconhecimento de padrões. Seguindo essa tendência, as pesquisas recentes em processamento de linguagem natural estão cada vez mais colocando foco na aplicação de novos métodos de aprendizado profundo. Por décadas, as abordagens de aprendizado de máquina direcionadas a problemas de PNL foram baseadas em modelos superficiais, por exemplo, máquinas de vetor de suporte (SVM) e regressão logística. O treinamento desses modelos exigia um alto nível de recursos. Nos últimos anos, as redes neurais baseadas em representações vetoriais densas vêm produzindo resultados superiores em várias tarefas de processamento de linguagem natural. *Deep learning* permite o aprendizado automático de representação de características em vários níveis. Em contraste, os sistemas tradicionais de NLP baseados no aprendizado de máquina (*machine learning*) desfrutaram de uma característica semiautomática (Young et al., 2018).

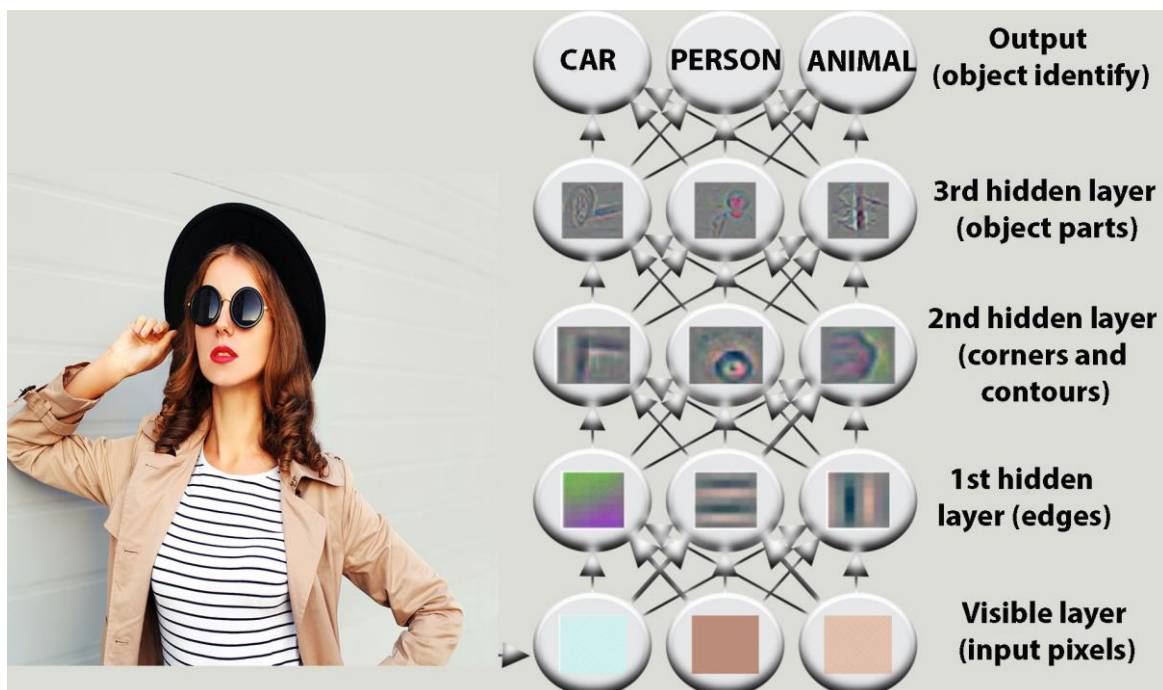
O aprendizado profundo refere-se à aplicação de redes neurais a grandes quantidades de dados para aprender um procedimento destinado a lidar com uma tarefa. A tarefa pode variar desde uma simples classificação a um raciocínio



complexo. Em outras palavras, o aprendizado profundo é um conjunto de mecanismos capazes de derivar uma solução ótima para qualquer problema, dado um conjunto de dados de entrada suficientemente extenso e relevante. De forma geral, o objetivo de *deep learning* é detectar e analisar estruturas ou características importantes, presentes nos dados, com o objetivo de formular uma solução para um determinado problema (Torfi et al., 2020).

O aprendizado profundo resolve o problema central no aprendizado de representação, introduzindo representações que são expressas em termos de outras representações mais simples. O aprendizado profundo permite que o computador construa conceitos complexos com base em conceitos mais simples.

Figura 1 – Um sistema de aprendizado profundo pode representar o conceito de imagem de uma pessoa combinando conceitos mais simples, como cantos e contornos, que por sua vez são definidos em termos de arestas



Fonte: Goodfellow; Bengio; Courville, 2016, p. 5.

Crédito: Rohappy/Shutterstock.

Deep Learning contempla em seu escopo de estudo uma série de arquiteturas que foram desenvolvidas em diferentes áreas de pesquisa. Por exemplo, em geral, as aplicações de NLP empregam *redes neurais recorrentes* (RNNs) e suas derivações; e as *redes neurais convolutivas* ou *convolucionais*



(CNNs) que são bastante utilizadas para reconhecimento visual profundo. Entretanto, CNN também tem sido aplicado dentro do escopo da NLP.

Os aplicativos de aprendizado profundo se baseiam nas escolhas de

- i. *representação de dados*; e
- ii. *do algoritmo de aprendizado profundo conjugado com a arquitetura*.

Para a representação de dados, surpreendentemente, em geral há uma disjunção entre quais informações são consideradas importantes para a tarefa em questão e qual representação realmente produz bons resultados. Por exemplo, na análise de sentimentos, a semântica do léxico, a estrutura sintática e o contexto são considerados por alguns linguistas como de importância primária. No entanto, estudos anteriores baseados no modelo vetorial (também referenciado como *saco de palavras – bag of words*) demonstraram desempenho aceitável, desde que lidando com grandes quantidades de dados. Esse modelo envolve uma representação que leva em conta apenas as palavras e sua frequência de ocorrência, ignorando a ordem e a interação das palavras e trata cada palavra como um recurso isolado.

O modelo vetorial desconsidera ainda a estrutura sintática, mas fornece resultados satisfatórios. Essa observação sugere que representações simples, quando combinadas com grandes quantidades de dados, podem funcionar tão bem ou melhor do que representações mais complexas. Isso auxilia a corroborar o argumento a favor da importância de algoritmos e arquiteturas de *deep learning* (Torfi et al., 2020).

Um desafio chave na pesquisa em NLP, em comparação com outros domínios como visão computacional, por exemplo, parece ser a complexidade de alcançar uma representação profunda da linguagem usando *modelos estatísticos*. Um objetivo da modelagem estatística de linguagem é a representação probabilística de sequências de palavras na linguagem. A tarefa primária em aplicações de NLP é fornecer uma representação de textos, tais como documentos. Isso envolve o *aprendizado de características*, ou seja, a extração de informações significativas para permitir o processamento e a análise dos dados brutos. Os métodos tradicionais começam com a demorada modelagem das características, por meio de uma cuidadosa análise humana de uma aplicação específica, e são seguidos pelo desenvolvimento de algoritmos para extrair e utilizar instâncias desses recursos. Por outro lado, os métodos de aprendizado supervisionado em *deep learning* são altamente orientados a dados

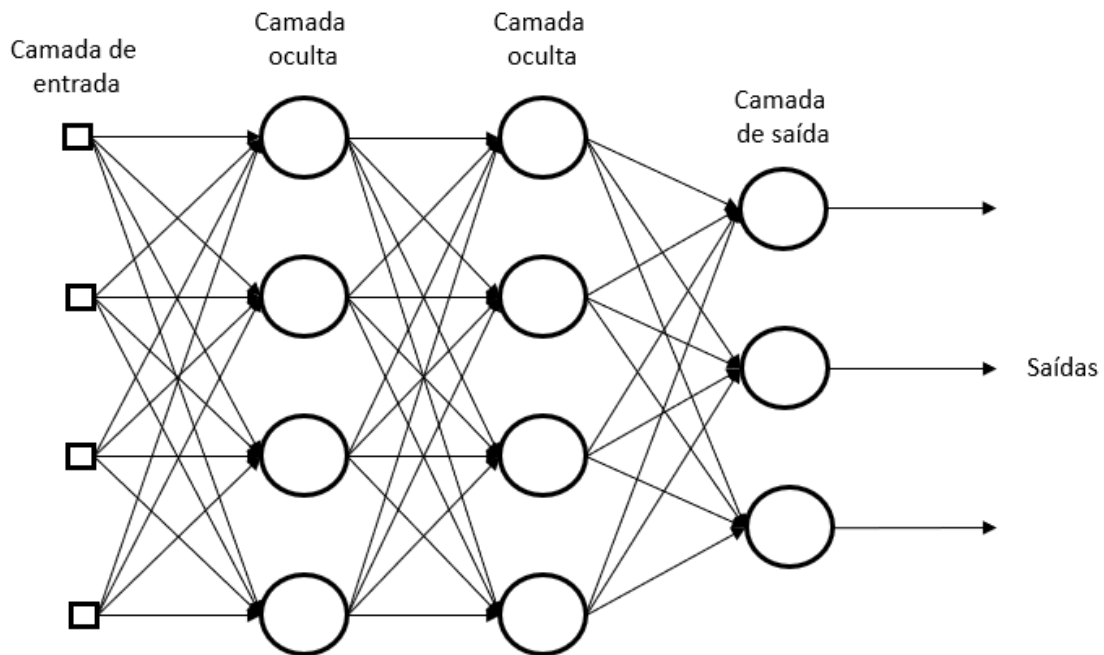


e podem ser usados em esforços mais gerais destinados a fornecer uma representação de dados robusta (Torfi et al., 2020).

1.1 *Perceptron* multicamada

Para o estudo das redes neurais para aprendizado profundo, é oportuno colocar como ponto de partida os conceitos relacionados com uma rede neural clássica e bastante explorada, que fornece a base para o entendimento de redes neurais mais complexas. Um *perceptron multicamada* (*Multilayer Perceptron* – MLP) é um tipo de rede neural artificial que possui pelo menos três tipos de camadas (camadas de entrada, ocultas e de saída). Uma camada é simplesmente uma coleção de neurônios operando para transformar informações da camada anterior para a próxima. Na arquitetura MLP, os neurônios em uma mesma camada *não se comunicam um com o outro*. Uma MLP emprega funções de ativação não lineares. Cada nó em uma camada se conecta a todos os nós na próxima camada, criando uma rede totalmente conectada (Figura 2). As MLPs são o tipo mais simples de *redes neurais alimentadas à frente* (*Feed-Forward Neural Networks* – FNN). As FNN representam uma categoria geral de redes neurais em que as conexões entre os nós não criam nenhum ciclo, ou seja, em um FNN não há ciclo de retroalimentação de informação (Torfi et al., 2020).

Figura 2 – Exemplo de um *perceptron* multicamada



Fonte: Haykin, 2001, p. 186.

As três características distintivas de um MLP sobre outras RNA podem ser destacadas a seguir (Haykin, 2001, p. 184):

1. O uso de *função de ativação não linear*: a diferença com a forma de ativação vista no modelo do *perceptron* de camada única é a mudança *suave* proporcionada pela ativação dos neurônios (A função *sin*al no algoritmo LMS causava uma mudança abrupta na ativação). Uma das funções mais utilizadas para se conseguir esse efeito no MLP é a função logística ou função sigmoide (Figura 3), com os valores de saída dentro da faixa $[0,1]$:

$$y_i = \frac{1}{1 + \exp(-v_j)}$$

Outra função de ativação bastante utilizada é a tangente hiperbólica, a qual, por sua vez, mantém os valores de saída na faixa $[-1,1]$:

$$y_i = \tanh v_j$$

O argumento v_j é calculado a partir das entradas x_k da rede MLP pelos pesos w_{jk} da camada considerada (mais os pesos constantes b_j):

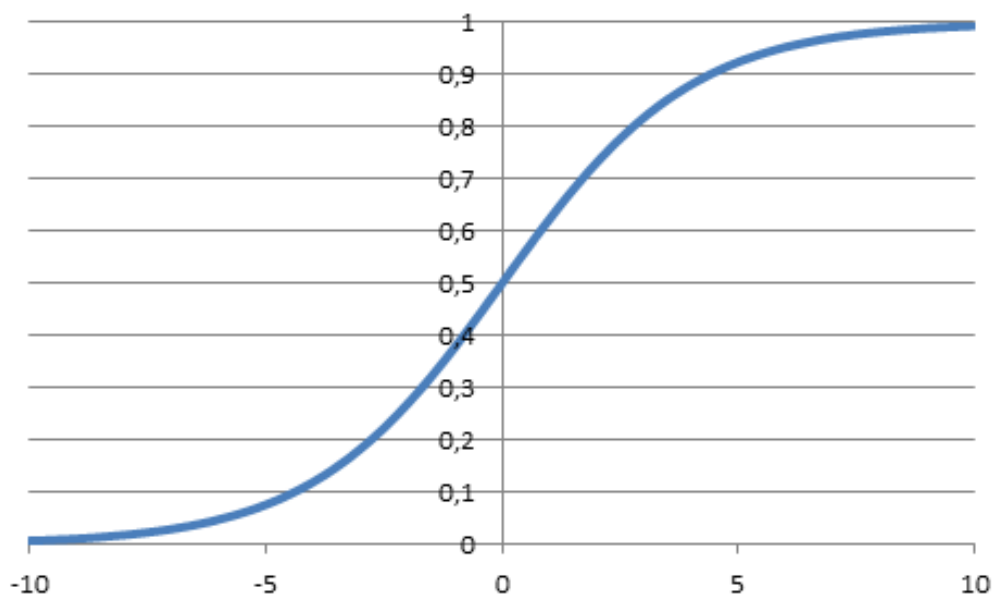


$$v_j = w_{jk}x_k + b_j$$

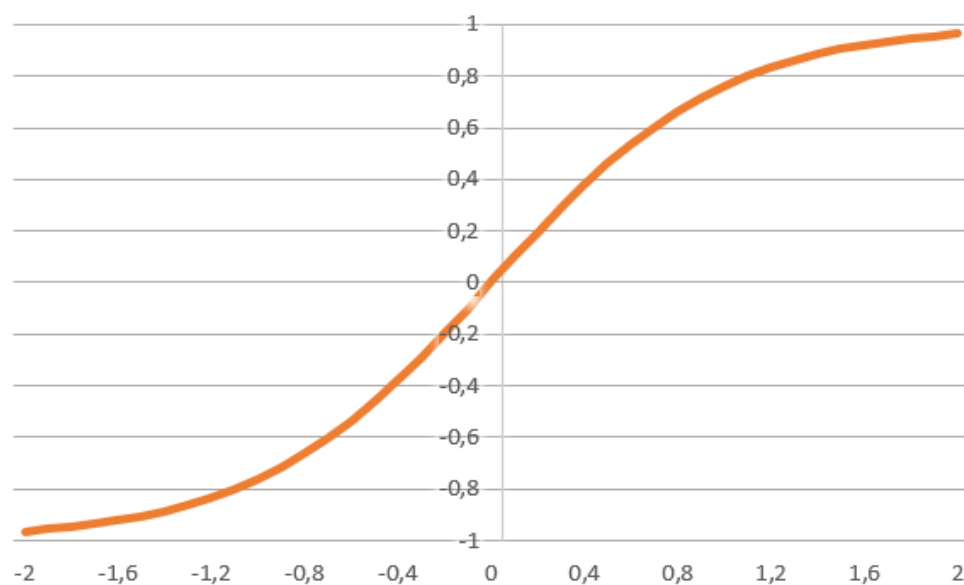
2. A existência de *camadas ocultas*, que não fazem parte nem da entrada nem da saída. As camadas ocultas podem aumentar consideravelmente o número de conexões na rede MLP.

3. O *alto grau de conectividade* que o MLP apresenta em função da sua arquitetura e da população de pesos sinápticos.

Figura 3 – Exemplos de função e ativação: (a) função sigmoide; (b) função tangente hiperbólica



(a)

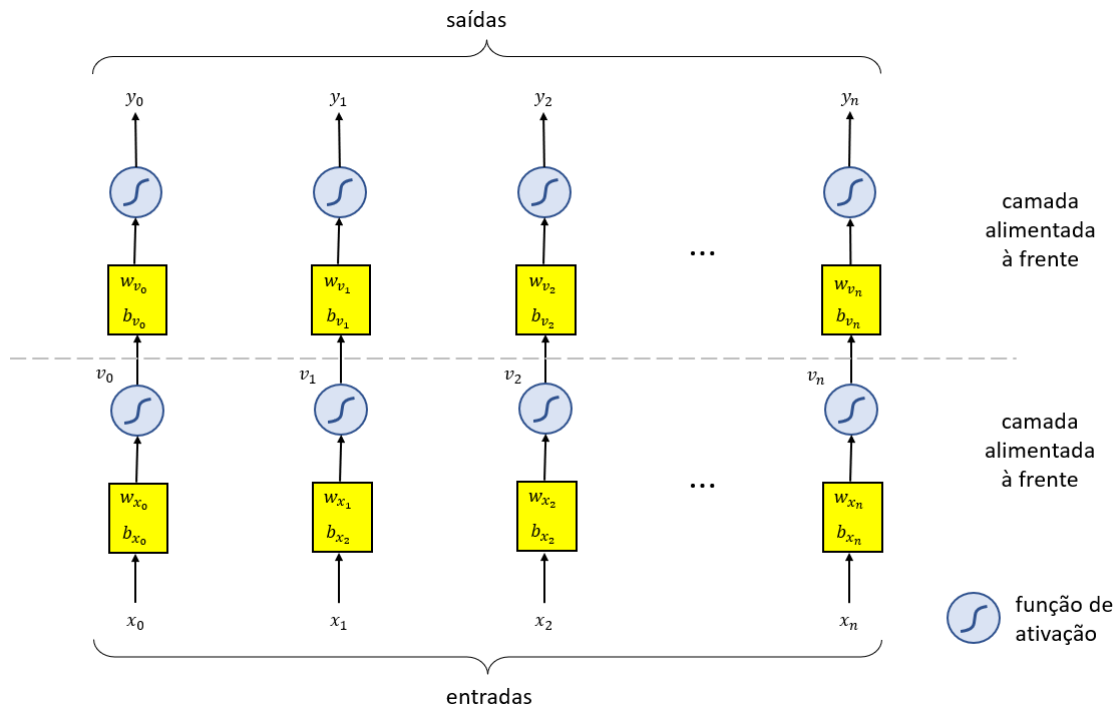


(b)



No exemplo da Figura 2, pode-se notar a quantidade de pesos sinápticos que surgem devido à presença de duas camadas ocultas. Contendo somente a camada de entrada e saída, o número de pesos sinápticos é de 12 pesos (4 entradas por 3 saídas). Com as duas camadas ocultas, o número de pesos é aumentado consideravelmente para 44 (4 entradas por 4 na primeira camada oculta – 16, mais 4 da primeira para 4 da segunda camada oculta – 16, mais 4 da camada oculta para 3 da camada de saída – 12). No entanto, a simples presença de mais camadas ocultas não significa necessariamente que ocorrerá melhoria de forma contínua na aprendizagem de um MLP.

Figura 4 – Representação esquemática genérica de uma rede MLP de duas camadas



Na Figura 4, há uma representação esquemática de uma rede MLP, onde se pode notar a existência de duas camadas alimentadas à frente (*feedforward*). Na representação não está explícito, porém uma MLP pode ainda conter mais camadas. De maneira geral, pode-se também afirmar a existência de uma camada de entrada de neurônios, sendo que a primeira camada alimentada à frente se caracteriza como uma camada oculta. Ainda que simplificado (pois pode haver um número diferente de entradas e saídas), esse diagrama é interessante para possibilitar a comparação com outras arquiteturas de redes



neurais utilizadas para NLP, nas quais se notará a presença dos elementos neuronais, composto dos pesos mais uma função de ativação.

Com relação ao aprendizado de uma rede MLP, é executada a *retroalimentação* de parte do sinal de saída, ajustando os valores dos pesos sinápticos, em direção ao menor erro. O *algoritmo de retropropagação de erro* (*error backpropagation*) possibilita que uma parte do sinal de saída da rede seja alimentada em sentido contrário das camadas (Haykin, 2001, p. 183). O algoritmo de retropropagação implementa dois tipos de retroalimentação:

- i. Uma da camada de saída à camada oculta posicionada anteriormente a ela; e
- ii. Outro tipo de camada oculta para camada oculta, até que se alcance a primeira camada com pesos sinápticos.

O algoritmo de retropropagação para o MLP envolve o processo chamado de *descida de gradiente*. Esse processo visa o cálculo do *gradiente local do erro* (a direção para onde tende a crescer o valor do erro médio calculado, calculado em função das derivadas parciais do erro em relação aos pesos), utilizando-o para corrigir os pesos sinápticos na direção contrária a este gradiente, em busca do erro mínimo local (Medeiros, 2018).

O entendimento de uma rede MLP fornece subsídios para a compreensão de outras redes que têm se tornado apropriadas para tarefas de maior complexidade. As redes neurais convolutivas, por exemplo, contém uma camada totalmente conectada ao final da sua arquitetura (*fully connected layer* – FCL), com as mesmas características de uma rede MLP. Nas redes neurais recorrentes, existem neurônios que seguem também a mesma lógica de funcionamento pesos-função de ativação, ainda que contenham elementos adicionais que aumentam a sua funcionalidade.

Saiba mais

Em linguagem Python, a API Keras, que é parte da biblioteca Tensorflow, implementa o modelo *Sequential*, que permite empilhar diferentes camadas em uma rede neural. A camada Dense permite adicionar uma camada ao estilo da MLP. Por exemplo, uma rede MLP com duas camadas de 10 neurônios cada pode ser implementada da seguinte forma:



```
model = keras.Sequential()  
model.add(layers.Dense(10, activation="sigmoid"))  
model.add(layers.Dense(10))
```

A camada Dense é bastante utilizada em diversos tipos de redes neurais, tais como as redes neurais convolutivas e as redes neurais recorrentes, onde é necessário implementar uma camada final para que se possa executar uma classificação.

Para conhecer a API Keras, acesse o *link* a seguir:

KERAS. Disponível em: <<http://keras.io>>. Acesso em: 10 mar. 2022.

1.2 Representação vetorial

O uso de redes neurais para tarefas que envolvam NLP requer tipos diferenciados de representação de dados de entrada que codifiquem as características das palavras de maneira a reduzir o espaço de entrada. Alguns exemplos de representação vetorial são os seguintes (Liu; Lin; Sun, 2020, p. 4-6):

- **Vetores *one-hot*:** o modo mais fácil de representar uma palavra de forma legível por computador, o qual possui a dimensão do tamanho do vocabulário e atribui 1 à posição correspondente da palavra e 0 às demais. É evidente que os vetores *one-hot* dificilmente contêm qualquer informação semântica sobre as palavras, exceto simplesmente distinguindo-as umas das outras;
- ***N-Gram*:** uma das primeiras ideias de aprendizagem de representação de palavras. Quando é necessário prever a próxima palavra em uma sequência, geralmente se olha para algumas palavras anteriores (e no caso de *n-gram*, elas são as $n - 1$ palavras anteriores). Considerando-se um *corpus* de grande escala, pode-se contar e obter uma boa estimativa de probabilidade de cada palavra sob a condição de todas as combinações de $n - 1$ palavras anteriores. Essas probabilidades são úteis para previsão de palavras em sequências e formam representações vetoriais para palavras, pois refletem os significados das palavras;
- ***Bag-Of-Words* (*bow*):** considera-se um documento como um *saco* de suas palavras, desconsiderando a ordem dessas palavras no documento.



Dessa forma, este pode ser representado como um vetor do tamanho do vocabulário. Cada palavra presente corresponde a uma dimensão única e diferente de zero. A seguir, uma pontuação pode ser calculada para cada palavra (por exemplo, o número de ocorrências) para indicar os pesos dessas palavras no documento. Os modelos BOW funcionam muito bem em aplicações como filtragem de spam, classificação de texto e recuperação de informações, provando que as distribuições de palavras podem servir como uma boa representação para o texto;

- **Word embeddings** (vetores de palavras de baixa dimensão): referem-se a métodos que incorporam palavras em representações distribuídas e usam o objetivo de modelagem da linguagem para otimizá-las como parâmetros do modelo. Exemplos populares são *word2vec*, *GloVe* e *fastText*. Apesar de haver diferenças entre eles em termos de detalhes, tais métodos são todos bastante eficientes para treinar, utilizam *corpora* em larga escala e foram amplamente adotados como incorporação de palavras em muitos modelos de NLP. As incorporações de palavras no *pipeline* de NLP mapeiam palavras discretas em vetores informativos de baixa dimensão e ajudam a esclarecer as redes neurais na computação e no entendimento de linguagens. Isso torna o aprendizado de representação uma parte crítica do processamento de linguagem natural.

Saiba mais

Em Python, o word embedding é feito por meio da API Keras, no modelo Sequential. Uma camada Embedding é adicionada a um modelo, por exemplo, da seguinte maneira:

```
model = keras.Sequential()  
model.add(layers.Embedding(input_dim=200, output_dim=32))
```

A camada *Embedding* (representação distribuída) faz a transformação dos valores de entrada de tamanho `input_dim`, em um conjunto de vetores de saída `output_dim`. A dimensão de entrada está relacionada com o tamanho do vocabulário.

A pesquisa sobre aprendizagem de representação em PNL deu um grande salto quando *ELMo* e *BERT* surgiram. Além de usar *corpora* maiores, existem mais parâmetros e mais recursos de computação, em comparação com o *word2vec*. Também levam em consideração contextos complicados dentro do



próprio texto. Isso significa que, em vez de atribuir a cada palavra um vetor fixo, *ELMo* e *BERT* usam redes neurais multicamadas para calcular representações dinâmicas para as palavras com base em seu contexto, o que é especialmente útil para palavras com vários significados (Liu; Lin; Sun, 2020).

TEMA 2 – REDES NEURAIS CONVOLUTIVAS

Após a popularização dos modelos vetoriais e sua capacidade de representar palavras em um espaço distribuído, surgiu a necessidade de uma função eficaz que extraia características de nível superior dos termos ou n-gramas. Isto possibilitaria a aplicação em várias tarefas de NLP, como análise de sentimentos, resumo, tradução automática e sistemas de pergunta e resposta (Q & A).

O estudo de RNA sobre a perspectiva do *deep learning* tem nas redes convolutivas um dos grandes desenvolvimentos para o reconhecimento de padrões. A concepção moderna de *redes convolutivas* ou *convolucionais* (*Convolutional Neural Networks* – CNN) inicia-se com LeCun et al (1998). As CNN constituem um tipo especializado de rede neural para processar dados que têm uma topologia conhecida, semelhante a uma grade. Como exemplos, podem ser utilizados *dados de séries temporais*, que podem ser considerados como uma grade unidimensional coletando amostras em intervalos de tempo regulares; e *dados de imagem*, que podem ser considerados como uma grade bidimensional de *pixels*.

Uma das diferenças essenciais com relação à abordagem do *perceptron* multicamada e modelos de redes neurais mais antigos refere-se ao uso da estratégia do *pré-treinamento intensivo de camadas* (*greedy layer-wise pre-training*), que pode melhorar a *performance* das RNA em problemas de alta complexidade (Bengio et al., 2007; Hinton, 2007).

As redes convolucionais têm sido extremamente bem-sucedidas em aplicações práticas. O nome *rede neural convolucional* indica que a rede emprega uma operação matemática chamada *convolução*, que é um tipo específico de operação linear. As redes convolucionais são simplesmente redes neurais que usam *convolução* no lugar da multiplicação geral da matriz em pelo menos uma de suas camadas (Goodfellow; Bengio; Courville, 2016, p. 330).

Uma *rede convolutiva* ou *convolucional* é composta de várias camadas que desempenham funções distintas, à medida que vai processando os valores



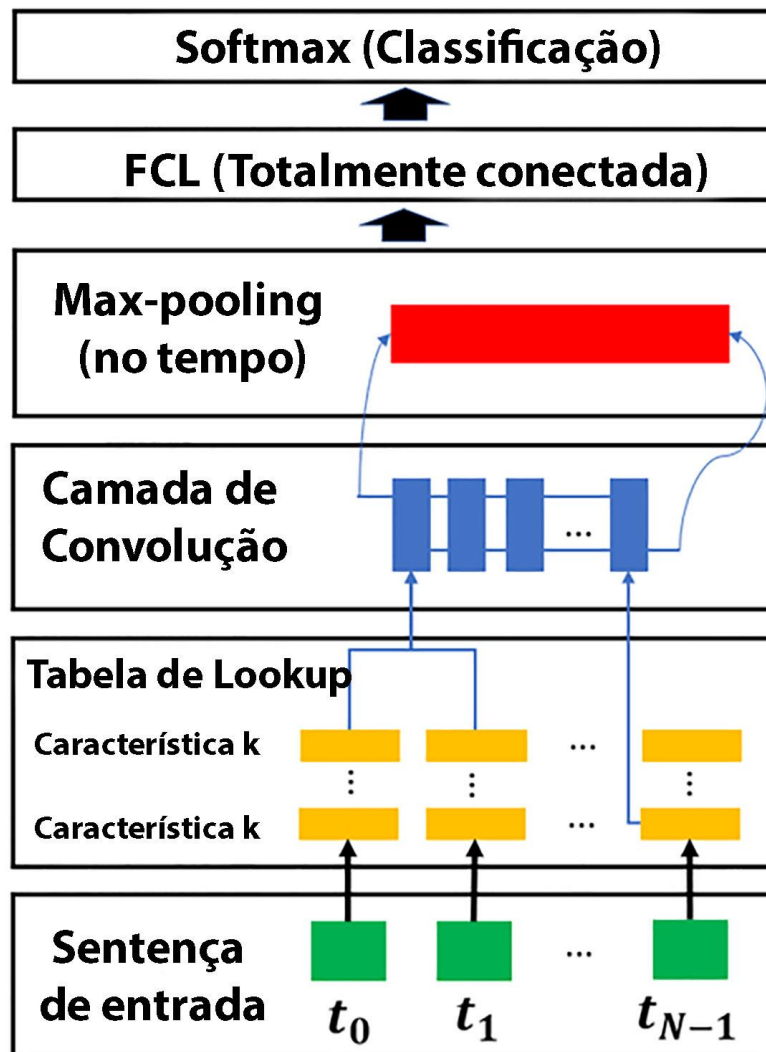
de entrada. Além das camadas que executam a convolução, existem outras que fazem a operação de *pooling* (que poderia ser traduzida como *agrupamento* ou *redução de resolução*), camadas com função de ativação específica como a *retificação linear* e camadas que fazem a poda (*dropout*). A camada final geralmente é uma camada totalmente conectada, a qual deve ser treinada de maneira similar ao MLP para fornecer os resultados.

O uso de redes convolutivas para modelagem de sentenças remonta a Collobert e Weston (2008), quando foi utilizado o aprendizado multitarefa para gerar múltiplas previsões para tarefas de NLP, como *tags* de entidade nomeada, palavras semanticamente semelhantes e um modelo de linguagem. Uma tabela de consulta (*lookup table*) foi usada para transformar cada palavra em um vetor de dimensões definidas pelo usuário. Dessa forma, uma sequência de entrada $\{s_0, s_1, \dots, s_{n-1}\}$ de n palavras foi transformada em uma série de vetores $\{t_0, t_1, \dots, t_{n-1}\}$ aplicando a tabela de consulta a cada uma de suas palavras (Figura 5).

Nota-se também na Figura 5 a presença de camadas que desempenham um papel essencial para as CNN, que são a *camada de convolução* propriamente dita, e a *camada max pooling*. Essas duas camadas podem ser combinadas aos pares para produzir a representação requerida para o problema em questão. Ao final de uma CNN, geralmente se encontra uma camada totalmente conectada (FCL) combinada com uma camada de classificação que utiliza a ativação *softmax*.



Figura 5 – Diagrama esquemático de uma rede convolucional e suas principais camadas para a classificação de palavras



Fonte: Collobert; Weston, 2008.

Saiba mais

Em Python, uma CNN pode ser implementada também por meio da API Keras (<http://keras.io>). Em um modelo *Sequential*, vários tipos de camadas podem ser empilhados: Conv2D (camada de convolução), MaxPooling2D (*max pooling*) e Flatten (transformação para entradas em uma rede totalmente conectada) e Dense. A seguir, é mostrado um exemplo simples para ilustrar uma CNN:

```
model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='softmax'))
```



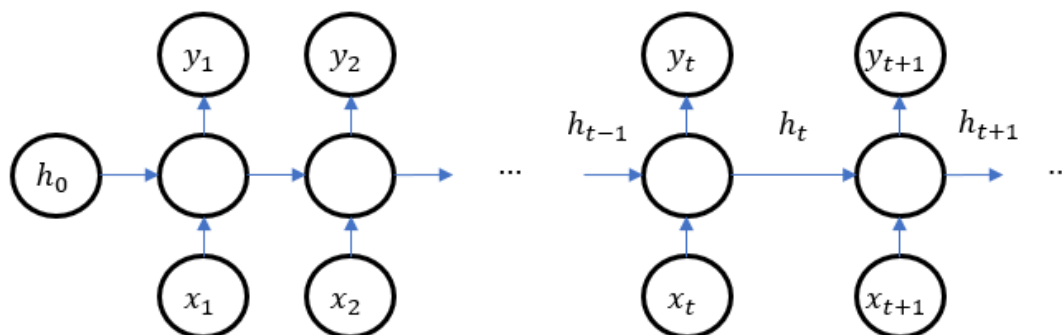
Para conhecer a ASPI Keras, acesse o *link* a seguir:

KERAS. Disponível em: <<http://keras.io>>. Acesso em: 10 mar. 2022.

TEMA 3 – REDES NEURAIS RECORRENTES

As *redes neurais recorrentes* (*Recurrent Neural Networks* – RNN) constituem uma família de redes neurais para processamento de dados sequenciais. Assim como uma rede convolucional é uma rede neural especializada para processar um reticulado de valores, tal como uma imagem, uma RNN é uma rede neural especializada para processar uma sequência de valores $x^1 \dots x^T$. Assim como as redes convolucionais podem escalar prontamente para imagens com grande largura e altura, e algumas redes convolucionais podem processar imagens de tamanho variável, as redes recorrentes podem escalar para sequências muito mais longas do que seria prático para redes sem especialização baseada em sequência. A maioria das redes recorrentes também pode processar sequências de comprimento variável (Goodfellow; Bengio; Courville, 2016).

Figura 6 – Representação esquemática de uma rede recorrente, mostrando os elementos ocultos para um processo de ativação específica



Fonte: Goodfellow; Bengio; Courville, 2016.

Na Figura 6, encontra-se uma representação de uma RNN simples. Os elementos x_i são as entradas da RNN, enquanto os elementos y_i são as saídas. A arquitetura de uma RNN pode ter diferentes configurações de entrada-saída, dependendo do objetivo desejado:

- **1-1:** uma entrada e uma saída. Ex: redes neurais simples.



- **1-N**: uma entrada e várias saídas. Ex.: sistemas de geração de música.
- **N-1**: várias entradas e uma saída. Ex.: análise de sentimento.
- **N-N**: várias entradas e várias saídas. Ex.: tradução automática.

A ativação em uma RNN acontece em dois momentos: um para atualizar os elementos internos h_t , e outra ativação para atualizar os elementos y_t :

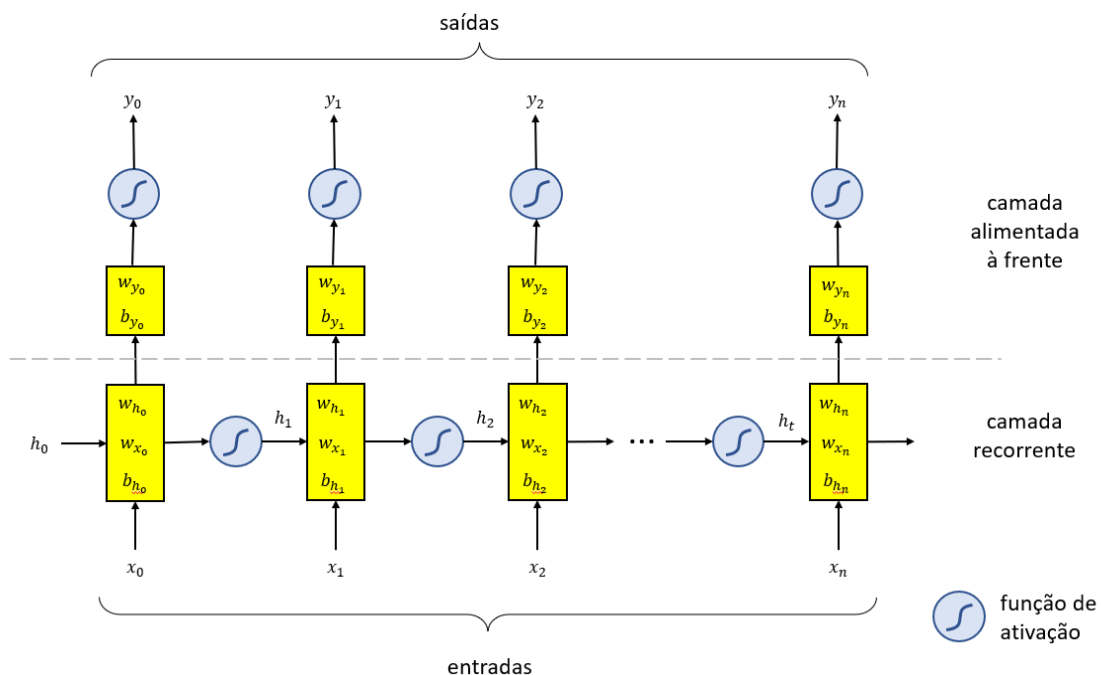
$$h_t = f_1(w_{hh}h_{t-1} + w_{hx}x_t + b_h)$$

e

$$y_t = f_2(w_{hy}h_t + b_y)$$

onde f_1 e f_2 são as funções de ativação, os pesos w_{hh} , w_{hx} e w_{hy} e os bias b_h e b_y são parâmetros compartilhados temporalmente. Na Figura 7 está a arquitetura genérica de uma RNN (note a diferença com relação à MLP apresentada na Figura 4).

Figura 7 – Representação de uma RNN, detalhando a camada recorrente da rede com os pesos respectivos e a ativação lateral, e a camada alimentada à frente (*feed-forward*)



Dado que uma RNN realiza o processamento sequencial modelando unidades em sequência, ela tem a capacidade de capturar a natureza sequencial



inerente presente na linguagem, em que as unidades são caracteres, palavras ou até mesmo frases. As palavras em um idioma desenvolvem seu significado semântico com base nas palavras anteriores na frase. Um exemplo simples afirmando isso seria a diferença de significado entre *cachorro* e *cachorro-quente*. As RNNs são feitas sob medida para modelar tais dependências de contexto em linguagem e tarefas de modelagem de sequência semelhantes, o que resultou em uma forte motivação para os pesquisadores usarem RNNs sobre CNNs nessas áreas (Young et al., 2018).

Outro fator que auxilia a aplicação de RNN para tarefas de modelagem de sequência está em sua capacidade de modelar comprimento variável de texto, incluindo frases muito longas, parágrafos e até documentos. Ao contrário das CNNs, as RNNs possuem etapas computacionais flexíveis que fornecem melhor capacidade de modelagem e criam a possibilidade de capturar contexto ilimitado. Essa capacidade de lidar com entradas de comprimento arbitrário tornou-se um dos pontos de venda de grandes obras usando RNNs (Young et al., 2018).

As redes neurais com camadas tais como o MLP são utilizadas quando existem um conjunto de entradas distintas e se espera a obtenção de uma classe ou um número real, em caso de regressão e assume-se que as entradas são todas independentes umas das outras. Já uma RNN é útil quando as entradas são sequenciais, sendo uma boa alternativa para a modelagem de problemas em NLP, pois esta lida com sequências de palavras e cada palavra é dependente das outras palavras. Para predizer qual será a próxima palavra em uma sequência, deve-se saber quais foram as palavras anteriores. Pode-se, então, resumir as vantagens de uma RNN (Goodfellow; Bengio; Courville, 2016):

- Permitem a manipulação de sequências de dados;
- Permitem a manipulação de entradas de comprimentos variáveis;
- Permitem o armazenamento de uma memória com relação aos dados de treinamento anteriores.

No entanto, dois tipos de problemas podem ser encontrados quando se faz o treinamento de uma RNN: a *dissipação de gradiente* (*vanishing gradient*) e a *explosão de gradiente* (*exploding gradient*). Como visto anteriormente, um gradiente é uma derivada parcial do erro com relação aos pesos. Ou seja, um gradiente mede o quanto a saída de uma função se modifica, em função das mudanças gradativas em suas entradas. O treinamento de uma rede envolve o



cálculo dos gradientes de erro, no sentido de encontrar no espaço de treinamento o nível de menor erro da rede neural.

A dissipação de gradiente acontece quando em uma RNN o valor do gradiente fica muito pequeno, próximo de zero. Com isso, o treinamento da RNN pode ser muito lento, ou mesmo não chegar a um valor razoável. No pior caso, o treinamento da RNN acaba sendo paralisado. Já a explosão de gradiente acontece o contrário: os valores assumidos pelo gradiente são tão grandes que o processo de treinamento termina por perder a estabilidade. Esses problemas se apresentam como limitações ao uso generalizado de uma RNN. Para dar conta de tais problemas, outras arquiteturas têm sido propostas (Young et al., 2018). Os modelos de sequência mais eficazes usados em aplicações práticas são chamados de *RNNs com portas (gated RNN)*, estando incluídas a LSTM e GRU.

Saiba mais

Em Python, uma RNN pode ser implementada também por meio da API Keras (<http://keras.io>). No modelo *Sequential*, a camada *SimpleRNN* pode ser empilhada a uma rede neural. A seguir, é mostrado um exemplo simples, que pode ser utilizado para aprendizado de série temporal:

```
model = Sequential()  
model.add(SimpleRNN(units=32, input_shape=(1,step),  
activation="sigmoid"))  
model.add(Dense(8, activation="relu"))  
model.add(Dense(1))
```

Para conhecer a API Keras, acesse o *link* a seguir:

KERAS. Disponível em: <<http://keras.io>>. Acesso em: 10 mar. 2022.

TEMA 4 – LSTM

A *memória de longo prazo (Long Short-Term Memory – LSTM)* refere-se a uma arquitetura diferenciada de RNN contendo um algoritmo de aprendizagem também baseado em descida de gradiente. LSTM foi projetado para superar os problemas que aparecem com as RNN, relativo à dissipação ou explosão de gradiente. Uma LSTM pode aprender a transpor intervalos de tempo em excesso, mesmo no caso de sequências de entrada incompressíveis e barulhentas, sem perda de recursos de intervalo de tempo curto (Hochreiter; Schmidhuber, 1997). LSTM tem sido bem-sucedida em aplicações como



reconhecimento de escrita, reconhecimento de fala, geração de escrita, análise de sentimento e tradução de máquina.

A característica principal de uma célula LSTM, além de conter portas de entrada e de saída, é a presença de uma porta de esquecimento (*forget gates*), que permite à LSTM contornar os problemas de dissipação ou explosão de gradiente. Na Figura 7, encontra-se uma representação esquemática de uma unidade LSTM, com o fluxo das informações sendo alimentado pelos elementos internos da rede. A representação se refere a uma célula apenas, sendo que uma rede é constituída por vários dessas células, conectadas entre si.

O estado interno da célula s_t e o estado oculto h_t são propagados de uma célula a outra dentro de uma rede LSTM. Cada célula é representada por uma entrada x_t , sendo que o próprio estado oculto h_t coincide também com a saída da célula y_t . A LSTM tem em sua estrutura diversas ativações, sendo originalmente utilizadas as funções sigmoide e a tangente hiperbólica (outros tipos de funções de ativação também têm sido utilizadas). A saída da porta de esquecimento é simbolizada por f_t e a atualização do estado da célula é feita pela porta de entrada i_t . O sinal c_t é um candidato à atualização do estado interno da célula, que interage com i_t , o qual é ativado por meio da função tangente hiperbólica. O sinal o_t se refere à porta de saída.

O conjunto de equações que forma a base da operação de uma célula LSTM é listado a seguir:

$$\mathbf{x} = [x_t \quad h_{t-1}]^T$$

$$f_t = \sigma(w_f \mathbf{x} + b_f)$$

$$i_t = \sigma(w_i \mathbf{x} + b_i)$$

$$c_t = \tanh(w_c \mathbf{x} + b_c)$$

$$s_t = f_t \odot s_{t-1} + i_t \odot c_t$$

$$o_t = \sigma(w_o \mathbf{x} + b_o)$$

$$h_t = o_t \odot \tanh(s_t)$$

$$y_t = h_t$$

Nesta descrição, σ se refere à função sigmoide, enquanto $\tanh$ refere-se à tangente hiperbólica. O operador $\odot$ é uma multiplicação e, junto com a operação soma, podem ser vistos também na Figura 8, de acordo com o fluxo previsto pelas equações.

Saiba mais

A multiplicação de Hadamard, que é diferente da multiplicação matricial, cuja operação é simplesmente multiplicar elemento a elemento nas mesmas linha e coluna de duas matrizes, produzindo uma terceira.

Figura 8 – Representação esquemática de uma célula LSTM. Podem-se notar as portas de entrada, saída e de esquecimento

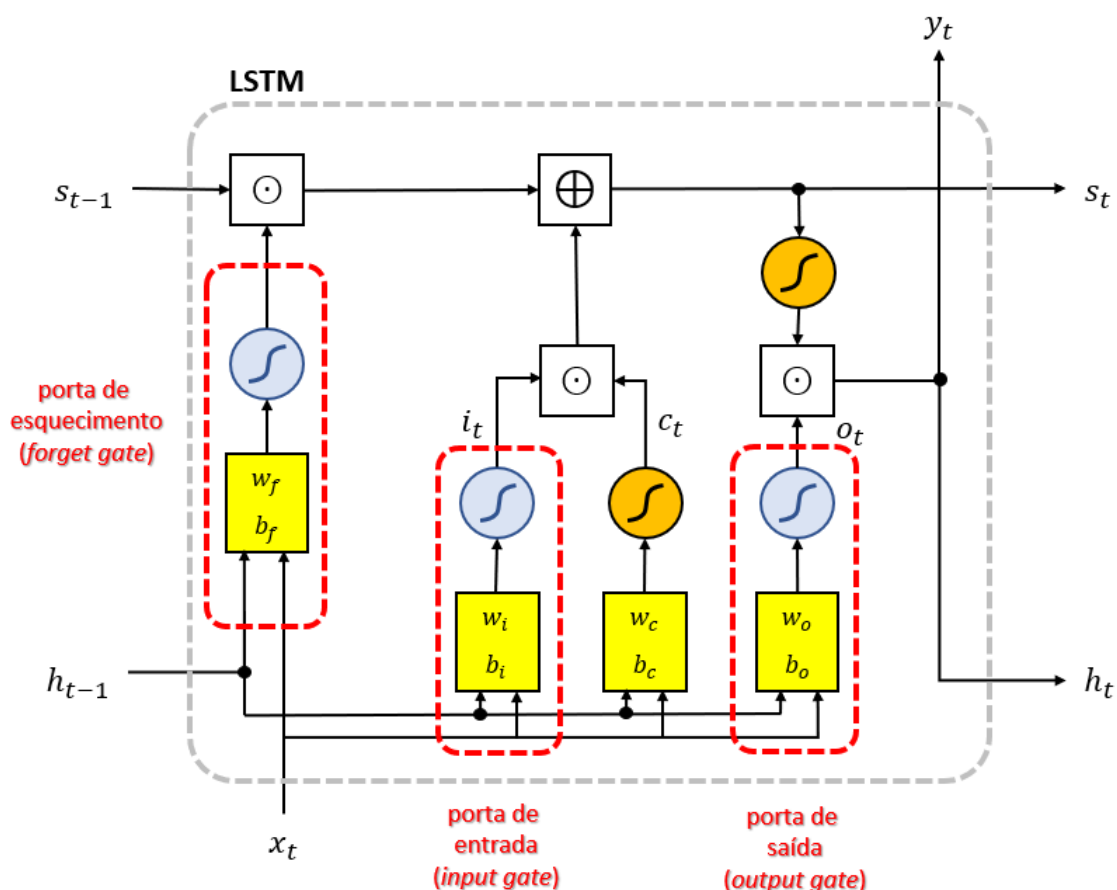
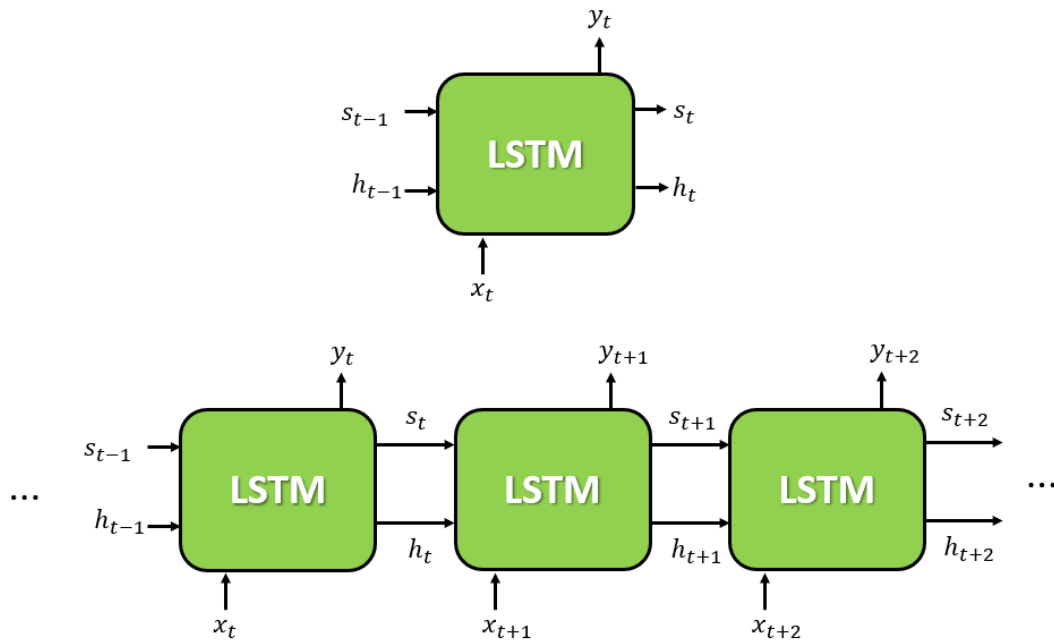
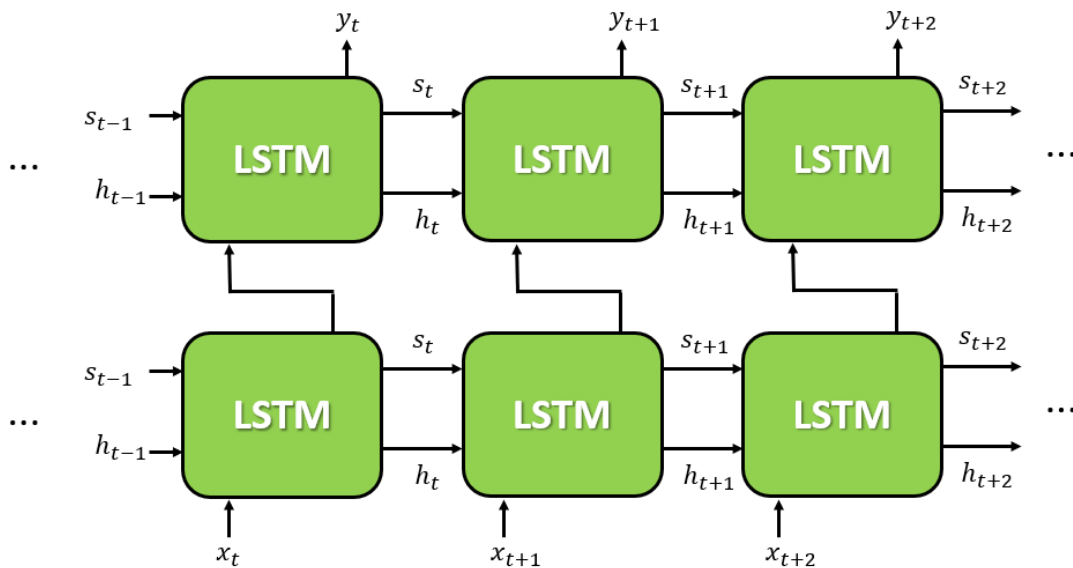




Figura 9 – Símbolo para uma célula LSTM e uma rede LSTM (a) e uma rede LSTM com duas camadas



(a)



(b)

Pode se notar na figura 7 o sinal oculto h_{t-1} se propagando célula a célula lateralmente, sendo alimentado com as entradas x_t para cada sistema de ativação (sigmóide e tangente hiperbólica) dentro da LSTM. O estado da célula s_{t-1} , que também é propagado lateralmente, é influenciado pela porta de



esquecimento f_t . O produto entre a entrada i_t e o sinal candidato c_t complementa o sinal para o próximo estado s_t . Por fim, o próximo estado oculto h_t , que também é o sinal de saída y_t , é obtido pela ativação desse estado s_t no produto com o sinal da porta de saída o_t . O uso da função sigmoide (que fixa os valores ativados na faixa $[0,1]$), balanceado com a função tangente hiperbólica (que, por sua vez, fixa os valores na faixa $[-1,1]$) proporciona à LSTM contornar o problema da explosão ou da dissipação de gradiente mencionada anteriormente, proporcionando estabilidade à sua operação.

Uma rede LSTM é composta de várias células encadeadas, como mostra a Figura 9. Redes LSTM com mais de uma camada também podem ser utilizadas, com a saída das células da camada anterior sendo alimentadas à camada seguinte. A rede *LSTM bidirecional* tem uma arquitetura ligeiramente diferenciada da mostrada na Figura 7 e proporciona a propagação de sinal lateralmente nos dois sentidos.

Saiba mais

Em Python, uma rede LSTM pode ser utilizada por meio da API Keras, no modelo *Sequential*. A camada LSTM pode ser adicionada a um modelo, por exemplo, da seguinte maneira:

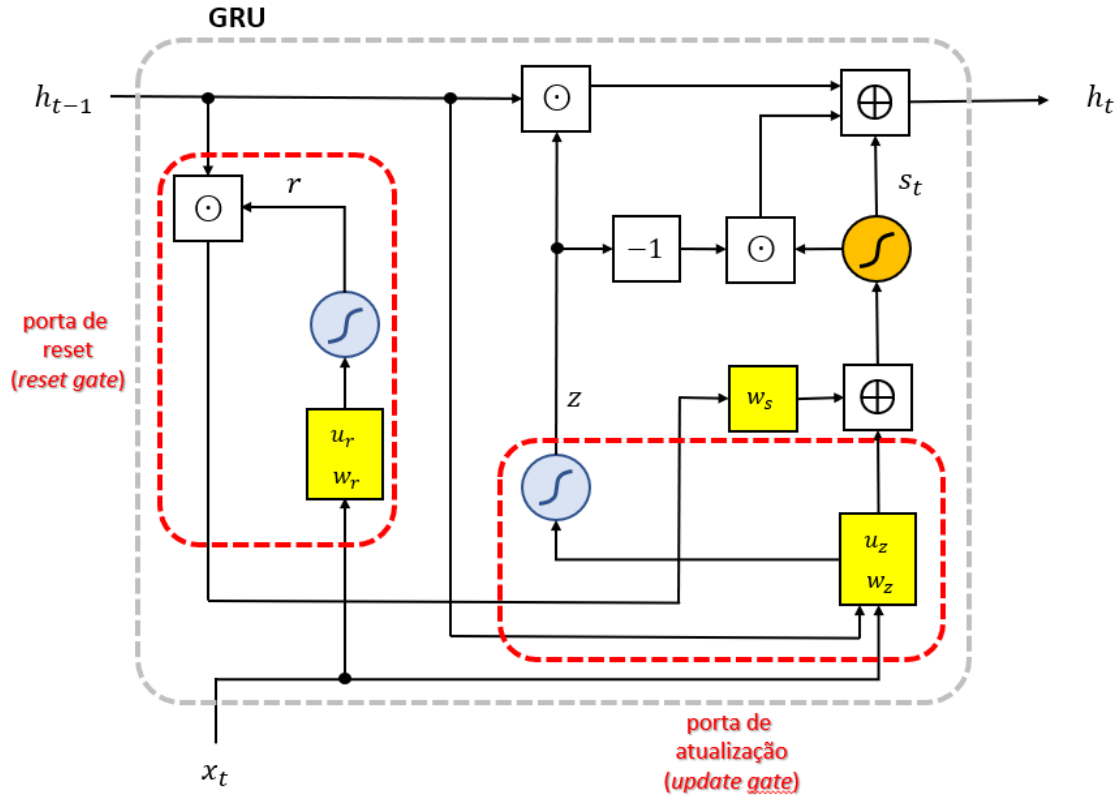
```
model = keras.Sequential()  
model.add(layers.Embedding(input_dim=1000, output_dim=64))  
model.add(layers.LSTM(128))  
model.add(layers.Dense(10))
```

A camada *Embedding* (representação distribuída) faz a transformação dos valores de entrada a partir de vetores utilizando *word embedding*.

TEMA 5 – GRU

As redes baseadas na *unidade recorrente com porta* (*Gated Recurrent Unit – GRU*) (Cho et al., 2014) possuem uma dinâmica de funcionamento semelhante à LSTM, porém com menor complexidade e tendo *performance* similar em muitas vezes. Uma célula GRU manipula o fluxo de sinais como uma unidade de memória, por meio de duas portas, a porta de *reset* (*reset gate*) e a porta de atualização (*update gate*). Com a propagação de apenas um sinal oculto lateralmente, uma GRU pode ser mais eficiente que uma rede RNN simples ou mesmo uma LSTM (Young et al., 2018).

Figura 10 – Representação esquemática de uma célula GRU. Estão assinaladas as portas de *reset* e de *atualização*



O conjunto de equações que forma a base da operação de uma célula GRU é listado a seguir:

$$z = \sigma(u_z x_t + w_z h_{t-1})$$

$$r = \sigma(u_r x_t + w_r h_{t-1})$$

$$s_t = \tanh(u_s x_t + w_s (h_{t-1} \odot r))$$

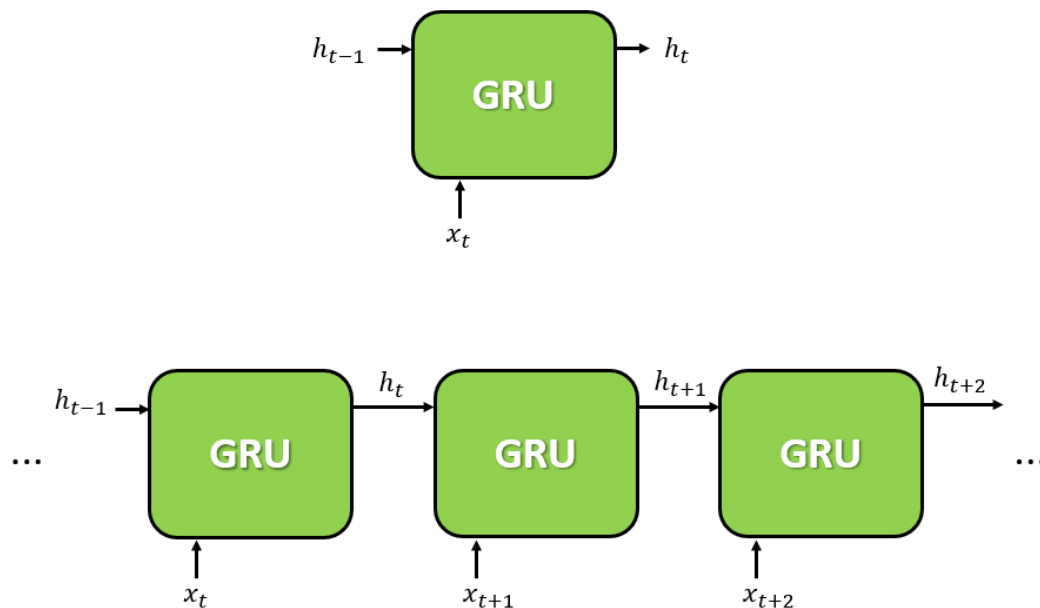
$$h_t = s_t - z \odot s_t + z \odot h_{t-1}$$

Pode-se notar na Figura 10 que uma célula GRU implementa dois tipos de pesos, u e w , que multiplicam a entrada x_t e o sinal oculto h_{t-1} , respectivamente. Por sua vez, as portas de reset r e de atualização z contêm um conjunto de pesos de acordo com esta implementação. A porta de reset r tem o objetivo de modular, por meio de um peso w_s , o sinal de atualização z , influenciando na produção do sinal h_t que é a saída e é utilizado para propagar o sinal para a próxima célula GRU. Da mesma forma que LSTM, uma célula GRU



possui os dois tipos de função de ativação sigmoide e tangente hiperbólica para contenção do sinal nas faixas de valores permitidos e evitar, assim, a instabilidade de operação devido à dissipação ou explosão de gradiente. Uma rede GRU é composta de várias células encadeadas, como mostra a Figura 11.

Figura 11 – Símbolo para uma célula GRU e uma rede GRU



Saiba mais

Em Python, uma rede GRU pode ser criada por meio da API Keras, dentro do modelo *Sequential*. A camada GRU pode ser adicionada a um modelo, por exemplo, da seguinte maneira:

```
model = Sequential()
model.add(Embedding(vocab_size, 256, input_length=max_caption_len))
model.add(GRU(output_dim=128, return_sequences=True))
model.add(Dense(128))
```

A camada *Embedding* faz a transformação dos valores de entrada a partir de vetores utilizando *word embedding*.

FINALIZANDO

O estudo atual de NLP e a elaboração de modelos e soluções para dar conta dos problemas reais relacionados às suas tarefas usam atualmente as ferramentas proporcionadas pelo aprendizado profundo. Tomando-se como base o *perceptron* multicamada, uma rede neural clássica, estudaram-se os tipos



de redes neurais no escopo de *deep learning*: as redes neurais convolutivas (CNN) e as redes neurais recorrentes (RNN). As RNN permitem que se trabalhe com sequências de palavras devidamente codificadas, permitindo um aprendizado com ativação lateral e transporte de sinal de um elemento oculto, que influencia o estado da unidade seguinte. Para resolver alguns problemas relacionados com RNN, foram elaboradas outras redes com LSTM e GRU. Essas unidades permitem que se trabalhe de forma mais eficiente com a memória do modelo, transmitindo os sinais ocultos e estados das células para as unidades seguintes. Com base nesses modelos, vários outros têm sido propostos, em busca de melhores abordagens e eficiência para modelos cada vez mais elaborados de NLP.



REFERÊNCIAS

- BENGIO, Y. et al. Greedy layer-wise training of deep networks. **Advances in Neural Information Processing Systems**, v. 19, n. 1, p. 153, 2007.
- CHO, K. et al. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In: EMNLP 2014-2014 CONFERENCE ON EMPIRICAL METHODS IN NATURAL LANGUAGE PROCESSING. **Proceedings of the Conference**, p. 1724-1734, 2014.
- COLLOBERT, R.; WESTON, J. A Unified architecture for natural language processing: deep neural networks with multitask learning. In: PROCEEDINGS OF THE 25TH INTERNATIONAL CONFERENCE ON MACHINE LEARNING. New York, Association for Computing Machinery, 2008. p. 160-167.
- GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. **Deep learning**. Cambridge-MA: MIT Press, 2016.
- HAYKIN, S. **Redes neurais: princípios e prática**. Porto Alegre: Bookman, 2001.
- HINTON, G. E. To recognize shapes, first learn to generate images. In: CISEK, P.; DREW, T.; KALASKA, J. F. (org.). **Computational Neuroscience: theoretical insights into brain function**. New York: Elsevier, 2007.
- HOCHREITER, S.; SCHMIDHUBER, J. Long Short-Term Memory. **Neural Computation**, v. 9, n. 8, p. 1735–1780, 1997.
- LECUN, Y. et al. Gradient-based learning applied to document recognition. **Proceedings of the IEEE**, v. 86, n. 11, p. 2278-2324, 1998.
- LIU, Z.; LIN, Y.; SUN, M. **Representation learning for natural language processing**. Singapore: Springer Singapore, 2020.
- MEDEIROS, L. F. de. **Inteligência artificial aplicada: uma abordagem introdutória**. Curitiba: InterSaberes, 2018.
- TORFI, A. et al. Natural language processing advancements by deep learning: a survey. **Arxiv**, v. 1, p. 1–23, 17 fev. 2020. Disponível em: <<http://arxiv.org/abs/2003.01200>>. Acesso em: 10 mar. 2022.
- YOUNG, T. et al. Recent trends in deep learning based natural language processing [Review Article]. **IEEE Computational Intelligence Magazine**, v. 13, n. 3, p. 55-75, 2018.